

Serie 10

Überlegen Sie sich bei allen folgenden Aufgaben, ob gewisse Eingabewerte zu Fehlern führen würden (z.B. Division durch Null) und verwenden Sie passende Fehlerkontrolle, um Laufzeitfehler abzufangen. Sie müssen nicht jedes Mal auf den richtigen Datentyp überprüfen.

Aufgabe 1:

Erstellen Sie eine Klasse `Polynom`. Dabei soll ein Polynom

$f(x) = a_0 + a_1x + \dots + a_nx^n$ erstellt werden, indem seine Koeffizienten als Liste `[a_0, ..., a_n]` übergeben (und abgespeichert) werden. Daraus soll der Grad (hier n) als weiteres Attribut abgespeichert und automatisch im Konstruktor berechnet werden. Schreiben Sie die folgenden Methoden für Ihre Klasse:

- `eval(self, x)` zur Auswertung an $x \in \mathbb{R}$, also soll der Wert $f(x)$ zurückgegeben werden.
- `deriv(self)`, welche die Ableitung (als Funktion) zurückgeben soll. Dabei soll der Rückgabewert ein neues Objekt der Klasse `Polynom` sein.
- `zeros(self)`, welche alle reellen Nullstellen berechnen soll, wenn das Polynom zweiten Grades ist. Falls der Grad nicht zwei ist, soll ein `ValueError` geworfen werden. Falls keine reellen Nullstellen existieren, soll `None` zurückgegeben werden.

Testen Sie Ihre Klasse und Methoden geeignet.

Aufgabe 2:

Erweitern Sie Ihre Klasse `Polynom` um folgende Methoden:

- eine magische Methode `__add__`, sodass `+` auf zwei Polynomen definiert ist und ein neues Objekt der Klasse erzeugt. Beachten Sie, dass die zu summierenden Polynome unterschiedliche Grade haben können.
- zwei magische Methoden `__mul__` und `__rmul__`, die die Multiplikation eines Polynoms mit Skalaren von rechts (`mul`, $f(x)\alpha$) bzw. links (`rmul`, $\alpha f(x)$) durchführen soll. Dabei soll jeweils ein neues Objekt der Klasse zurückgegeben werden. Achten Sie darauf, was passiert, wenn Sie mit `0` multiplizieren.
- zwei magische Methoden `__mul__` und `__rmul__`, die die Multiplikation von zwei Polynomen bestimmen soll. Adaptieren Sie dafür die Methoden zur Multiplikation mit Skalaren, indem Sie eine `isinstance`-Abfrage hinzufügen.
Hinweis: Die Koeffizienten sind durch die Cauchy-Produktformel gegeben. Achten Sie auf den Fall, dass eines der Polynome das Nullpolynom ist.

Testen Sie Ihre Klasse und Methoden geeignet.

Aufgabe 3:

Erweitern Sie Ihre Klasse `Polynom` um folgende Methoden:

- eine magische Methode `__str__`, die das Polynom als `string` ausgibt. Dabei soll es in der Form $a_0 + a_1x + \dots + a_nx^n$ ausgegeben werden, wobei Terme mit 0-Koeffizienten weggelassen werden sollen, also $1 + x^2$ anstatt $1 + 0x + x^2$.
- zwei magische Methoden `__le__` und `__ge__`, die nach betragsmäßigem Wachstum entscheiden sollen:
 - Ein Polynom mit höherem Grad ist größer, und bei gleichem Grad n entscheidet der Absolutbetrag der n -ten Koeffizienten.
 - Sind auch diese gleich, so wollen wir hier Gleichheit.

Testen Sie Ihre Klasse und Methoden geeignet.

Aufgabe 4:

Erstellen Sie eine Klasse `MySet`, die dieselben Funktionalitäten hat wie der `set` Datentyp. Ihr Konstruktor soll eine Liste so abändern, dass keine Duplikate mehr vorhanden sind. Schreiben Sie die folgenden Methoden für Ihre Klasse:

- `__add__`, welche die Vereinigung $\cup$ mittels `+` implementieren soll. Achten Sie darauf, dass auch dann keine Duplikate vorkommen dürfen.
- `__sub__`, welche die Mengendifferenz $\setminus$ mittels `-` implementieren soll. Achten Sie darauf, dass nicht alle Elemente der einen Menge auch in der anderen vorkommen könnten.
- `add(self, a)`, welche das Element `a` zum gegebenen Objekt hinzufügt. Achten Sie auch hier auf Duplikate.

Sie dürfen in dieser Aufgabe **keine** Funktionen für `set`s verwenden.

Aufgabe 5:

Erstellen Sie eine Klasse `TimeInterval`, die zwei Zahlen `start`, `end` zur Initialisierung nimmt und speichert. Ihr Konstruktor soll eine Fehlermeldung ausgeben, wenn `end < start`. Schreiben Sie (zwei bis) drei magische Methoden, sodass folgendes Verhalten sichergestellt ist:

- Bei `print(TimeInterval(start, end))` soll die Ausgabe in der Form `[start, end]` sein. (Sofern `start ≤ end`.)
- Die Abfrage `TimeInterval(a, b) < TimeInterval(c, d)` soll `True` liefern, wenn `a < c`, also das erste Intervall vor dem anderen anfängt. Ansonsten soll `False` herauskommen. (Sofern `a ≤ b` und `c ≤ d`.)
- Für eine Liste `lst` von Objekten der Klasse `TimeInterval` soll `print(sorted(lst))` eine sortierte Liste von Intervallen ausgeben.
 - Bsp.: Für `lst = [TimeInterval(1,5), TimeInterval(2,3)]`,

`TimeInterval(0,4)`] soll `[[0, 4], [1, 5], [2, 3]]` ausgegeben werden.

- *Hinweis:* Hier brauchen Sie die `__repr__` Methode und nicht `__str__` .

Aufgabe 6:

Gegeben sei ein lineares Gleichungssystem der Form

$$\begin{array}{rcl} u_{1,1}x_1 + \cdots + u_{1,n}x_n & = & b_1 \\ \vdots & & \vdots \\ u_{n,1}x_1 + \cdots + u_{n,n}x_n & = & b_n \end{array}$$

mit einer oberen Dreiecksmatrix $U = (u_{i,j})_{i,j=1}^n$, wo also $u_{i,j} = 0$ für $i > j$ gilt. Durch die Dreiecksstruktur kann das Gleichungssystem effizient mit Rückwärtseinsetzen gelöst werden:

1. Berechne $x_n = b_n / u_{n,n}$ und dann
2. für i von $n - 1$ bis 1 berechne

$$x_i = \frac{1}{u_{i,i}} \left(b_i - \sum_{j=i+1}^n u_{i,j} x_j \right).$$

Schreiben Sie eine Funktion `back_substitution(U, b)` , die eine obere Dreiecksmatrix `U` sowie einen Vektor `b` erhält und den Lösungsvektor `x` zurückgibt.

Achten Sie insbesondere auf mögliche Laufzeitfehler und fangen Sie diese gezielt ab. Zeigen Sie weiters, wie Ihr Code mithilfe des Python Debuggers auf Fehler getestet werden kann.